中国科学院大学
《计算机组成原理(研讨课)》实验报告

姓名 黄静怡 学号 2024K8009907042 专业 计算机科学与技术
实验项目编号 1 实验名称 处理器基本功能部件设计——寄存器堆和算术逻辑单元

注 1:	撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下(注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中 "-" 为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
注 2:	使用 `git add` 及 `git commit` 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
注 3:	实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

一、 逻辑电路结构与仿真波形的截图及说明(比如 Verilog HDL 关键代码段{包含注释}及其对应的逻辑电路结构图{自行画图, 推荐用 PPT 画逻辑结构框图后保存为 PDF, 再插入到 LaTeX. 中}、相应信号的仿真波形和信号变化的说明等)

- **寄存器堆设计。**

  关键代码:

```
`timescale 10 ns / 1 ns

`timescale 10 ns / 1 ns

`define DATA_WIDTH 32
`define ADDR_WIDTH 5

module reg_file(
  input                    clk,
  input [`ADDR_WIDTH - 1:0] waddr,
  input [`ADDR_WIDTH - 1:0] raddr1,
  input [`ADDR_WIDTH - 1:0] raddr2,
  input                    wen,
  input [`DATA_WIDTH - 1:0] wdata,
  output [`DATA_WIDTH - 1:0] rdata1,
  output [`DATA_WIDTH - 1:0] rdata2
);

  // TODO: Please add your logic design here
  // 定义32个32位寄存器
  reg [`DATA_WIDTH-1:0] regs [2**`ADDR_WIDTH - 1:0];
```

```
  // 异步读操作 组合逻辑
  // 0号寄存器恒输出0
  assign rdata1 = (raddr1 == {`ADDR_WIDTH{1'b0}}) ? {`DATA_WIDTH{1'b0}} : regs[raddr1];
  assign rdata2 = (raddr2 == {`ADDR_WIDTH{1'b0}}) ? {`DATA_WIDTH{1'b0}} : regs[raddr2];

  // 同步写操作 时钟上升沿触发
  always @(posedge clk) begin
    // 写使能有效 且 不是0号寄存器时才写入
    if (wen && (waddr != {`ADDR_WIDTH{1'b0}})) begin
      regs[waddr] <= wdata;
    end
  end

endmodule
```

寄存器堆的逻辑结构可分为三部分:

寄存器阵列:32 个 32 位寄存器组成的存储核心,是数据的物理载体。

读通路:2 组多路选择器,根据读地址 raddr1/raddr2 选择对应寄存器输出,同时通过比较器判断地址是否为 0,强制输出 0。

写通路:由写使能 wen 和地址非 0 判断共同控制的写使能逻辑,在时钟上升沿将 wdata 写入目标寄存器,0 号寄存器被屏蔽写入。

逻辑电路图:

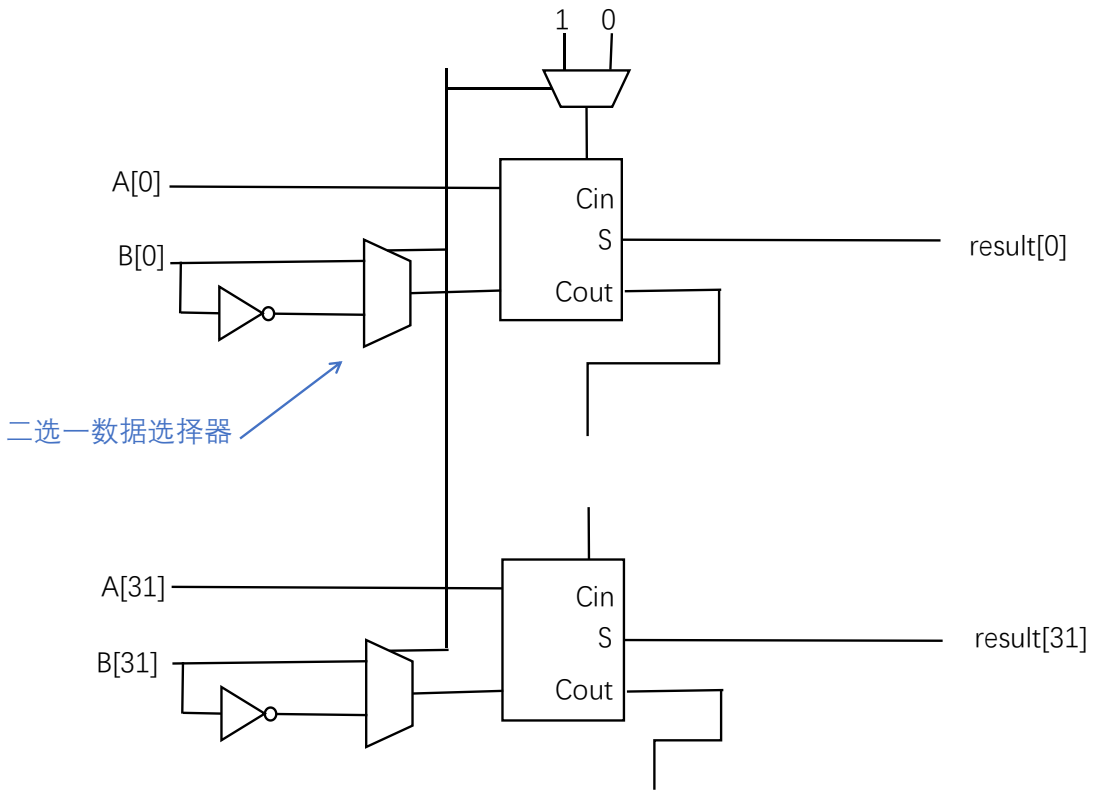

图 1: 寄存器堆逻辑电路图

仿真波形图：

观察并分析波形可以发现，只有当 clk 上升沿中，wen 为 1 时，waddr 对应的寄存器才会被写入数据 wdata。

而读数据时与 clk 无关，属于组合逻辑电路，另外在这部分波形中也可以体现本项目中当读地址为 0 时，读出的数据为 0 的特性。

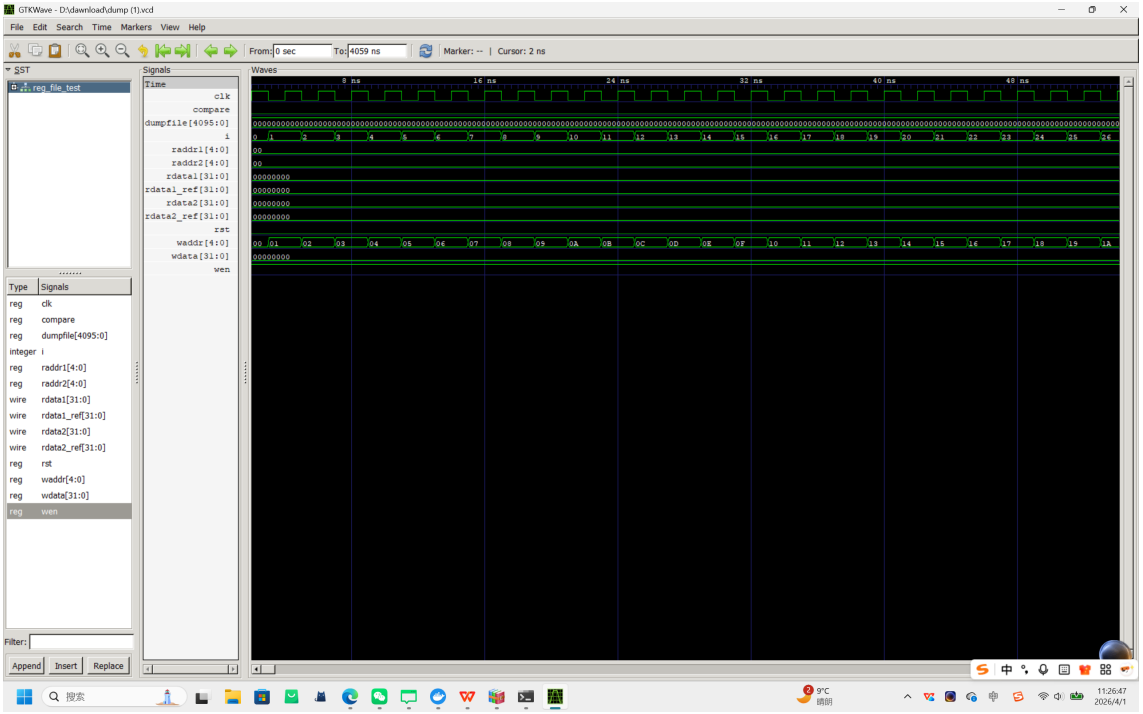

图 2: 寄存器堆仿真波形图

- ALU **设计**。

```
`timescale 10 ns / 1 ns

`define DATA_WIDTH 32

module alu(
   input [`DATA_WIDTH - 1:0] A,
   input [`DATA_WIDTH - 1:0] B,
   input [            2:0] ALUop,
   output                  Overflow,
   output                  CarryOut,
   output                  Zero,
   output [`DATA_WIDTH - 1:0] Result
);

wire [`DATA_WIDTH - 1:0] B_mux;
wire op_sub;
wire [`DATA_WIDTH - 1:0] sum;
wire cout_tmp;
wire overflow_tmp;
wire slt_result;
```

```
// 减法控制
assign op_sub = (ALUop == 3'b110) || (ALUop == 3'b111);
assign B_mux = op_sub ? ~B : B;

// 32位加法器 (ADD/SUB/SLT 复用)
assign {cout_tmp, sum} = A + B_mux + op_sub;

// 溢出判断
assign overflow_tmp = (A[31] == B_mux[31]) && (sum[31] != A[31]);
assign Overflow = ((ALUop == 3'b010) || (ALUop == 3'b110)) ? overflow_tmp : 1'b0;

// SLT 公式: 有符号比较必须 = 溢出 ^ 符号位
assign slt_result = overflow_tmp ^ sum[31];

assign Result = (ALUop == 3'b000) ? A & B : // AND
              (ALUop == 3'b001) ? A | B :   // OR
              (ALUop == 3'b010) ? sum :     // ADD
              (ALUop == 3'b110) ? sum :     // SUB
              (ALUop == 3'b111) ? {{31{1'b0}}, slt_result} : // SLT
              {`DATA_WIDTH{1'b0}};

// 零标志
assign Zero = (Result == {`DATA_WIDTH{1'b0}});

assign CarryOut = (ALUop == 3'b010) ? cout_tmp : // ADD
                (ALUop == 3'b110) ? ~cout_tmp : // SUB
                1'b0;                           // 其他

endmodule
```

本 ALU(算术逻辑单元)是 RISC-V 单周期 CPU 的核心运算部件,支持 5 种基础运算,并输出溢出、进位、零标志。模块输入为两个 32 位操作数 A、B,3 位操作控制码 ALUop;输出为 32 位运算结果 Result,以及 3 个状态标志 Overflow(溢出)、CarryOut(进位 / 借位)、Zero(零标志)。

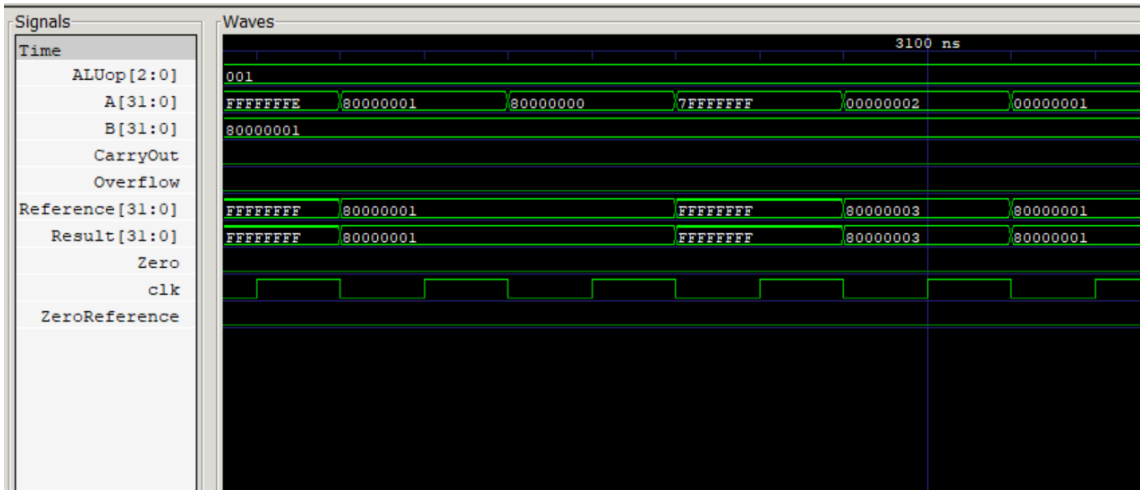

图 3: ALUOR 模块波形图

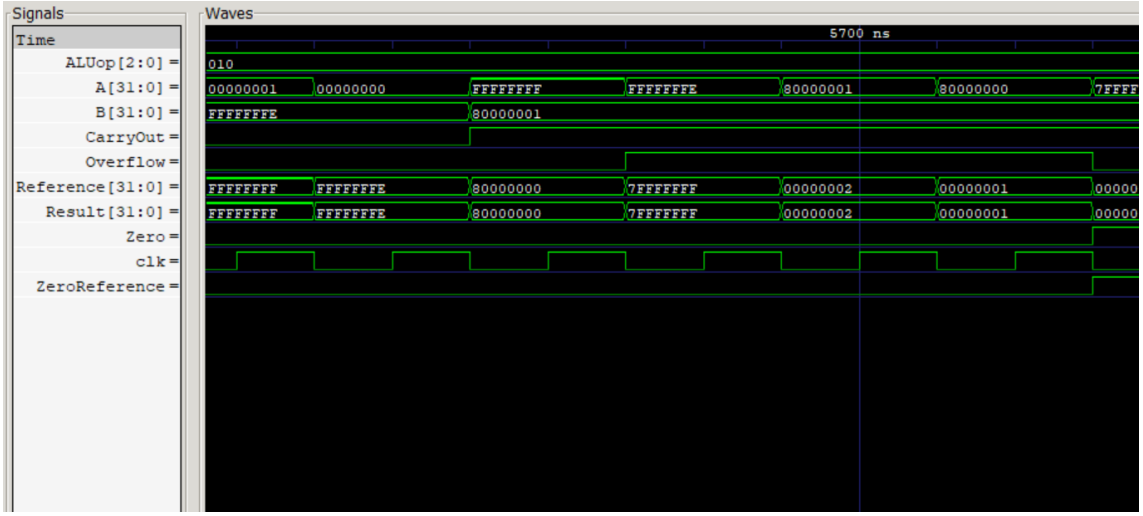

图 4: ALUADD 模块波形图

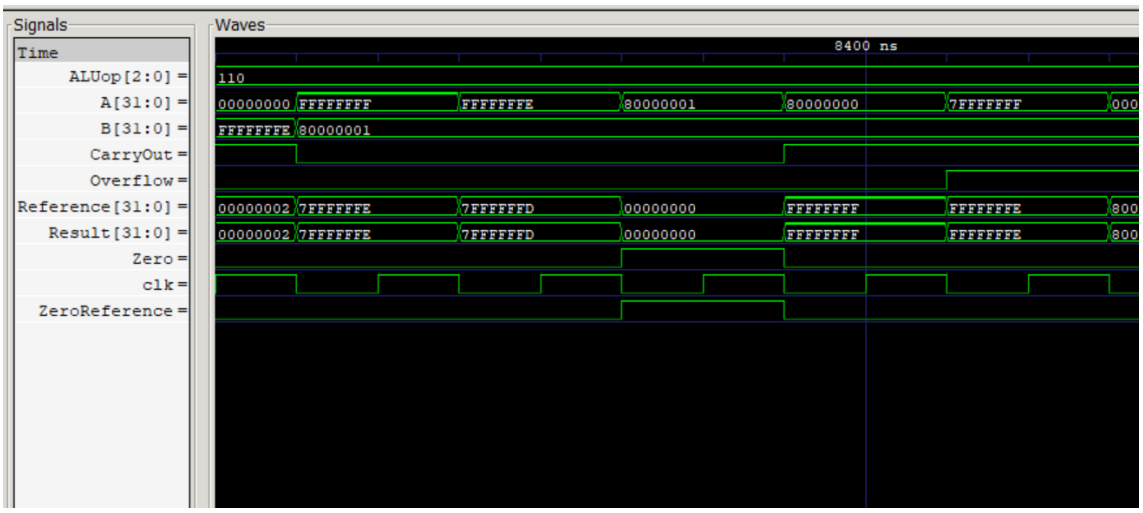

图 5: ALUSUB 模块波形图

选取了几个模块进行分析,可以看出基本符合我们的要求

二、 实验过程中遇到的问题、对问题的思考过程及解决方法(比如 RTL 代码中出现的逻辑 bug,逻辑仿真和 FPGA 调试过程中的难点等)

- **寄存器堆设计问题。**

```
// 定义32个32位寄存器
reg [`DATA_WIDTH-1:0] regs [2**`ADDR_WIDTH - 1:0];
```

这种定义方法可以直接定义 32 个 32 位寄存器总体上没有遇到什么问题,只要搞清楚含义即可完成

- ALU **设计问题。**

在一开始,我尝试使用了 always 模块来写,但是发现存在生成组合逻辑环的潜在风险,组合逻辑环会导致电路延时无法确定、时序无法收敛,随后进行修改,咨询了豆包,最后使用 assign 完美完成。

```
// 减法控制
assign op_sub = (ALUop == 3'b110) || (ALUop == 3'b111);
assign B_mux = op_sub ? ~B : B;
```

输入 A=ffffffff、B=00000000、ALUop=2(ADD)时,运算结果输出全为未定义的 x,与参考模型 ffffffff 不符。未定义值 x 是 Verilog 仿真中信号赋值冲突或未正确驱动的典型表现,排查代码发现最初的加法器设计中对进位输入 cin 进行了多级嵌套赋值,导致信号驱动逻辑冲突,组合逻辑无法生成有效输出。于是我简化加法器核心逻辑,删除冗余的进位中间信号,通过 op_sub 单一控制信号实现加法和减法的统一驱动,直接用 cout_tmp, sum = A + B_mux + op_sub 实现 32 位加法器,消除信号赋值冲突,确保所有输入信号均被正确驱动,未定义值问题解决。

```
// 32位加法器 (ADD/SUB/SLT 复用)
assign {cout_tmp, sum} = A + B_mux + op_sub;
```

在 AND/OR 等逻辑运算时,Overflow 仍有随机非 0 输出,虽实验规定该场景下 Overflow 为"未定义",但代码未做显式置 0 处理,存在逻辑冗余。Overflow 是专属于有符号数加减法的标志位,仅在 ALUop=2(ADD)和 ALUop=6(SUB)时有效,其他运算场景下应显式置 0,既符合组合逻辑"所有输出必须有确定值"的设计要求,也能避免仿真中出现无意义的信号波动。在 Overflow 的赋值逻辑中增加操作码判断,仅当执行 ADD/SUB 操作时输出溢出判断结果 overflowtmp,其余操作直接置 0,使代码逻辑更严谨,仿真信号更稳定。

```
// 溢出判断
assign overflow_tmp = (A[31] == B_mux[31]) && (sum[31] != A[31]);
assign Overflow = ((ALUop == 3'b010) || (ALUop == 3'b110)) ? overflow_tmp : 1'b0;
```

输入 A=7fffffff、B=ffffffff、ALUop=7(SLT)时,代码输出 Result=00000001,参考模型为 00000000,有符号比较结果判定错误。SLT 的核心是判断有符号数 A<B,其结果由 A-B 的减法结果推导,但最初代码直接将减法结果的最高位(符号位)作为 SLT 输出,未考虑有符号数减法溢出的情况;当减法运算发生溢出时,结果的符号位会发生反转,直接使用会导致比较结果错误,正确的 SLT 结果应通过溢出标志与减法结果符号位异或得到。定义临时信号 slt_result,通过 assign slt_result=overflow_tmp^sum[31] 实现溢出修正后的有符号比较,再将该结果扩展为 32 位作为 SLT 操作的输出,修复后比较结果完全匹配参考模型。

```
// SLT 公式: 有符号比较必须 = 溢出 ^ 符号位
assign slt_result = overflow_tmp ^ sum[31];
```

•

三、 实验所耗时间

在课后,你花费了大约____6____ 小时完成此次实验。